Class Notes: Session 4 - Different Approaches to Git (Workshop)

Overview

Session 4 focuses on practical, hands-on applications of Git concepts from Sessions 3 and 4, specifically remote branches and pulling in Git. This workshop emphasizes real-world scenarios where developers collaborate using remote repositories, manage branches, and synchronize changes. The activities are designed to reinforce branching, merging, and remote operations, aligning with the deliverables mapped to *Version Control with Git and GitHub* (OnlineVarsity, Sessions 3 & 4).

Remote Branches in Git

Remote branches are pointers to the state of branches in a remote repository (e.g., on GitHub). They allow developers to track and collaborate on changes hosted remotely.

- **Key Concepts**:
  - **Remote Branch Naming**: Remote branches are prefixed with the remote name (e.g., `origin/feature-login`).
  - **Tracking Branches**: Local branches can track remote branches to simplify pushing and pulling.
  - **Commands**:
    - List remote branches: `git branch -r`.
    - Create a local branch tracking a remote branch: `git checkout --track origin/<branch-name>`.
    - Push a local branch to remote: `git push origin <branch-name>`.
    - Delete a remote branch: `git push origin --delete <branch-name>`.
- **Example**:
  A developer creates and pushes a branch to a remote repository:

```
git checkout -b feature-user-profile
echo "User profile page" > profile.html
git add profile.html
git commit -m "Added user profile page"
git push origin feature-user-profile
git branch -r  # Lists origin/feature-user-profile
```

- **Use Case**:
  A team working on a web application pushes feature branches to a remote repository, allowing team members to review and test changes before merging.
- **Scenario**:
  A developer creates a `feature-checkout` branch locally, pushes it to GitHub for team review, and tracks the remote branch to stay updated with collaborators' changes.
- **Source**: *Version Control with Git* by Jon Loeliger and Matthew McCullough.

Pulling in Git

Pulling retrieves updates from a remote repository and integrates them into the local repository. It combines `git fetch` (downloads changes) and `git merge` (integrates changes into the current branch).

- **Commands**:
  - Pull changes: `git pull origin <branch-name>`.
  - Fetch without merging: `git fetch origin`.
  - View fetched changes: `git log --oneline --all`.
- **Example**:
  A developer pulls updates from the `main` branch:

  ```
  git checkout main
  git pull origin main
  ```

  This fetches and merges changes from `origin/main` into the local `main` branch.
- **Use Case**:
  A developer pulls the latest changes from a team's shared repository to ensure their local codebase is up-to-date before starting new work.
- **Scenario**:

A student pulls updates from a professor's repository to get the latest assignment instructions and sample code before beginning their work.

- **Source**: *Git* by Ryan Hodson.

Workshop Activities

The following activities reinforce the concepts of remote branches and pulling, building on branching and merging from Session 3.

Activity 1: Creating and Pushing a Remote Branch

- **Objective**: Practice creating a local branch, making changes, and pushing it to a remote repository.
- **Steps**:
  i. Initialize a new Git repository or use an existing one.
  ii. Create a branch: `git checkout -b feature-contact-form`.
  iii. Add a file `contact.html` with basic HTML content.
  iv. Commit the changes: `git commit -m "Added contact form page"`.
  v. Push to remote: `git push origin feature-contact-form`.
  vi. Verify the remote branch: `git branch -r`.
- **Example Output**:

```
git branch -r
# Output: origin/feature-contact-form
```

- **Scenario**:
  A team member creates a branch for a new feature, pushes it to GitHub, and shares the branch name with colleagues for code review.

Activity 2: Pulling and Merging Remote Changes

- **Objective**: Practice pulling changes from a remote branch and resolving potential conflicts.
- **Steps**:
  i. Clone a repository (e.g., a shared class repository).
  ii. Switch to the `main` branch: `git checkout main`.

iii. Pull updates: `git pull origin main`.
iv. Create a new branch: `git checkout -b update-readme`.
v. Edit `README.md` and commit: `git commit -m "Updated README"`.
vi. Pull again to check for conflicts: `git pull origin main`.
vii. If conflicts arise, resolve them manually, stage, and commit.

- **Example Conflict Resolution**:
  If `README.md` has conflicts:

```
git pull origin main
# Edit README.md to resolve conflicts
git add README.md
git commit -m "Resolved merge conflicts in README"
```

- **Scenario**:
  A developer pulls the latest changes from `origin/main` to incorporate a teammate's updates before merging their own branch.

Activity 3: Managing Remote Branches

- **Objective**: Practice tracking and deleting remote branches.
- **Steps**:
  i. Track a remote branch: `git checkout --track origin/feature-contact-form`.
  ii. Make a small change and commit: `git commit -m "Updated contact form styling"`.
  iii. Push changes: `git push origin feature-contact-form`.
  iv. Merge the branch into `main` locally and push:
  `git checkout main; git merge feature-contact-form; git push origin main`.
  v. Delete the remote branch: `git push origin --delete feature-contact-form`.
- **Example Output**:

```
git push origin --delete feature-contact-form
# Output: Deleted branch feature-contact-form (was abc1234).
```

- **Scenario**:
  After completing a feature, a developer merges it into `main`, pushes the updates, and deletes the remote branch to keep the repository clean.

Classwork

1. **Create and Push a Remote Branch**:
   - Initialize a new repository or use an existing one.
   - Create a branch named `feature-footer`.
   - Add a file `footer.html` with footer content.
   - Commit and push the branch to a remote repository (e.g., GitHub).
   - Verify the remote branch with `git branch -r`.
2. **Pull Remote Changes**:
   - Clone a public repository (e.g., `https://github.com/octocat/Spoon-Knife`).
   - Pull the latest changes from the `main` branch.
   - Create a new branch `update-docs`, edit a file, and commit.
   - Pull again to check for updates and resolve any conflicts.
3. **Manage Remote Branches**:
   - Track a remote branch (e.g., `origin/feature-contact-form`).
   - Make a change, commit, and push to the remote branch.
   - Merge the branch into `main` and delete the remote branch.

Session Test

1. **Multiple Choice**:
   - What does `git pull origin main` do?
     - A) Pushes local changes to the remote repository
     - B) Fetches and merges changes from the remote `main` branch
     - C) Deletes the remote `main` branch
     - D) Creates a new local branch
     - **Answer**: B
2. **Short Answer**:
   - Explain the difference between `git fetch` and `git pull` in the context of remote branches.
3. **Practical**:
   - Create a branch named `feature-sidebar`, add a file `sidebar.css` with some CSS code, commit it, and push it to a remote repository. Verify the remote branch exists.
4. **True/False**:

- A remote branch can be deleted using `git branch -d <branch-name>`. (False)

5. **Scenario-Based**:
   - Describe how a developer can collaborate with a team by creating a feature branch, pushing it to a remote repository, pulling updates from `main`, merging their branch, and cleaning up the remote branch.